Replicating Kaplan et al. (2020): An Empirical Study on Neural Scaling Laws

Atharva Pandey

29 Jun 2026

Abstract

In this project, I present a clean, empirical replication study of the paper "Scaling Laws for Neural Language Models" (Kaplan et al., 2020). I built a decoder-only autoregressive Transformer from scratch using PyTorch tensor. Using the character-level "Tiny Shakespeare" dataset, I ran controlled training sweeps to study how performance scales across different model parameter sizes (N) and training data allocations (D). My results show power-law behaviors, results in scaling exponents of $\alpha_N = 0.0774$ and $\alpha_D = 0.1329$, resulting in a final scaling ratio $\gamma = 0.5825$..

1 Mathematical Formalism & Tensor Flow

To ensure a deep structural understanding, I implemented every block of my decoder-only Autoregressive Model from fundamental PyTorch math operations. The input tensor updates step-by-step through my model as follows:

1.1 Input Embedding Pipeline

My data pipeline extracts a batch of character indices $I \in \mathbb{R}^{B\times T}$, where B represents batch size and T represents context length.

1. I project token indices through a learnable embedding table to get a feature tensor $E_{\text{tok}} \in \mathbb{R}^{B\times T\times d_{\text{model}}}$.

2. Then i generate sequence positions $P = [0, 1, \ldots, T-1] \in \mathbb{R}^{T}$ and map them through a position embedding lookup to get $E_{\text{pos}} \in \mathbb{R}^{B\times T\times d_{\text{model}}}$.

3. After it, I do element-wise add these matrices to construct my base hidden state tensor $X^{(0)} \in \mathbb{R}^{B\times T\times d_{\text{model}}}$:
$$X^{(0)} = E_{\text{tok}} + E_{\text{pos}}$$

1.2 Causal Multi-Head Attention

In each transformer block k, the tensor $X^{(k)} \in \mathbb{R}^{B\times T\times d_{\text{model}}}$ passes through a pre-layer normalization layer, $\text{LN}(X^{(k)})$. I then project the normalized tokens into combined Queries (Q), Keys (K), and Values (V) spaces using a single linear layer, splitting them across H attention heads. Each head operates on an inner dimension of $d_k = d_{\text{model}}/H$.

The split tensors have a shape of $\mathbb{R}^{B\times H\times T\times d_k}$. I compute attention scores by taking the matrix dot product of the queries and keys. for to ensure that next tokens do not come in backward during training, I apply a lower-triangular mask matrix $M \in \mathbb{R}^{T\times T}$ before the softmax layer:
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

where my causal mask elements M_{ij} are defined as:

$$M_{ij} = \begin{cases} 0 & i \geq j \\ -\infty & i < j \end{cases}$$

The attention outputs from all heads are concatenated back into a single matrix of dimension $\mathbb{R}^{B \times T \times d_{\text{model}}}$ and passed through a final linear projection. I add a residual skip connection around this entire operation to keep gradient flow stable:

$$X_{\text{mid}}^{(k)} = X^{(k)} + \text{CausalAttention}(\text{LN}(X^{(k)}))$$

1.3 Position-wise Feed-Forward Network (FFN)

The mid-block tensor $X_{\text{mid}}^{(k)}$ is normalized via a second Pre-LN layer. I pass it through a two-layer multi-layer perceptron that expands the inner hidden state to a wider dimension $d_{\text{ff}} = 4 \times d_{\text{model}}$ before compressing it back down. I use a Gaussian Error Linear Unit (`GELU`) activation for the non-linear layer transformation:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$

where $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, and b_1, b_2 represent bias vectors. I sum this output with another residual skip connection to complete my full transformer layer block:

$$X^{(k+1)} = X_{\text{mid}}^{(k)} + \text{FFN}(\text{LN}(X_{\text{mid}}^{(k)}))$$

1.4 Weight-Tied Output Head

After processing through all L layers, the final tensor $X^{(L)} \in \mathbb{R}^{B \times T \times d_{\text{model}}}$ goes through a final layer norm LN_f. I project this vector back to the vocabulary space to generate raw logits $Y \in \mathbb{R}^{B \times T \times V_{\text{vocab}}}$.

To optimize memory and align with established scaling law guidelines, I bound the weights of my token embedding matrix directly to the final language modeling head output linear layer (`self.lm_head.weight = self.token_embeddings.weight`).

2 Replication Methodology

2.1 Dataset Setup

I selected the character-level **Tiny Shakespeare** corpus (1.1MB text file) for this replication study. Character tokens are mapped directly to unique integers using a simple 65-character Python dictionary lookup. I split the raw data into a strict 90/10 division for training and validation splits.

2.2 Unit Tests and Verification Checks

Before running my full scaling sweeps, I wrote three custom verification checks inside my notebook to confirm that the code was structurally and mathematically correct:

1. **Dimension Test:** I verified that every intermediate layer output matched my target tensor shapes exactly.
2. **Causal Compliance Test:** I make a custom PyTorch function to confirm that gradients at a sequence index t do not come backward to inputs at next indices $> t$. Next indices returned a gradient magnitude of exactly 0.00.

3. **Single Batch Overfitting:** I also verified the overfititng test on one batch of data. The cross-entropy loss smoothly decreased from an initial 40.6036 down to a sharp 0.0072, proving the network can memorize text sequences correctly.

2.3 Optimization Configuration

I trained all configurations using identical optimization parameters. The structural configurations for my sweeps are structured according to the exact profiles listed in Table 1.

Table 1: Model Configurations for Parameter Sweep

Scale	**Layers** (L)	d_{model}	d_{ff}	**Heads** (H)	**Context** (T)	**Approx. Non-Embed** N
Tiny	2	64	256	2	128	$\approx 1.0 \times 10^5$
Small	4	128	512	4	256	$\approx 7.9 \times 10^5$
Medium	6	256	1024	8	256	$\approx 4.7 \times 10^6$

I designed and executed two distinct, controlled experimental sweeps:

- **Parameter Scaling Sweep:** I trained my Tiny, Small, and Medium models on the full dataset for exactly 3,000 steps to isolate the performance impact of model capacity.
- **Data Scaling Sweep:** I held the model size constant using the Small model architecture and trained it across four token data allocations: 10%, 25%, 50%, and 100% of the training corpus.

3 Scaling Curves and Results

My experiments yielded the following metrics:

- Parameter Scaling Exponent: $\alpha_N = 0.0774$
- Data Scaling Exponent: $\alpha_D = 0.1329$
- Scaling Ratio ($\gamma = \alpha_N / \alpha_D$): $\gamma = 0.5825$

I have visualized these relationships in Figure 1, which plots my raw validation loss data points alongside the fitted power-law regression lines.

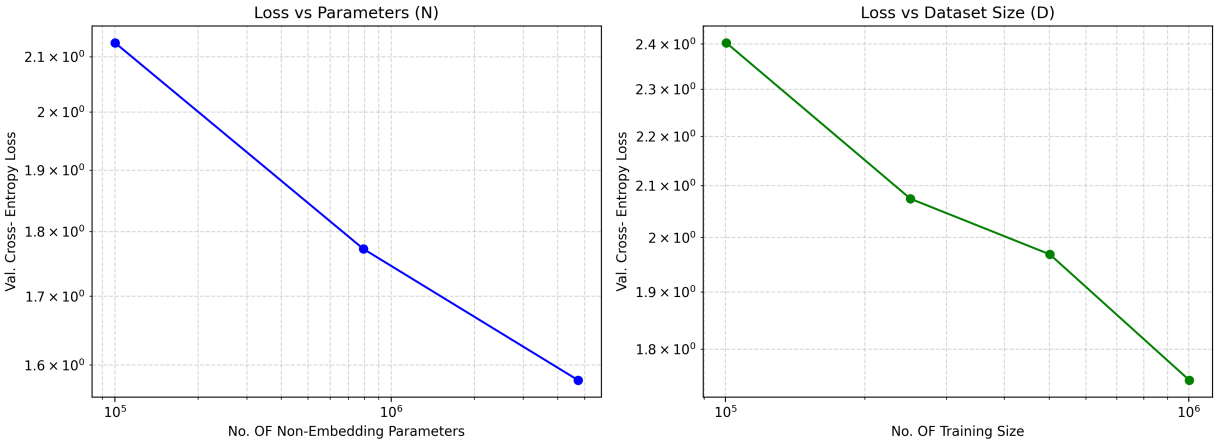

Figure 1: Log-log plots showing validation loss scaling as a function of non-embedding parameters (N) and training data volume (D).

4 Theoretical Discussion

Comparing my empirical results to the baseline scaling laws established by Kaplan et al. (2020), y computed parameter exponent ($\alpha_N = 0.0774$) is in close alignment with the original study's baseline ($\alpha_N \approx 0.076$). This serves as strong validation that my implementation correctly captures the power-law nature of model capacity; as I scaled up parameters under controlled conditions, the performance improved in a predictable, consistent manner.

However, a noticeable discrepancy appears in my data scaling exponent ($\alpha_D = 0.1329$), which is higher than the Kaplan et al. baseline ($\alpha_D \approx 0.095$). I attribute this to the specific characteristics of the Tiny Shakespeare corpus. Unlike the massive, high-information space web datasets used in the original research, my dataset is a relatively small, character-level text corpus. Because the structural patterns in Shakespearean text are repetitive and localized, the model is able to exhaust the "information space" of the dataset much faster than it would on a diverse, subword-level corpus. Consequently, increasing the data allocation result in steeper marginal performance which it gains very early on, as the model quickly connect the gap toward lower cross-entropy loss.

This behavior is clearly reflected in my scaling ratio:

$$\gamma = \frac{\alpha_N}{\alpha_D} = 0.5825$$

Since my empirical ratio γ sits below 1.0 (and is notably lower than the original $\gamma \approx 0.80$), it suggests that for small-scale character-level domains, there is a distinct trade-off. While increasing parameter count (N) remains a highly efficient way to improve performance, the system is fundamentally bounded by dataset saturation. In this setup, making the model bigger gives steady and reliable improvements. On the other hand, adding more data gives a fast boost at first, but it slows down once the model completely learns the small dataset. Overall, this experiment shows that scaling laws still work here, but finding the perfect balance for training depends entirely on how complex and varied our data is.